

October 16

Problem 1: Top K Frequent Elements

Problem Statement:

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Link to problem:

<https://leetcode.com/problems/top-k-frequent-elements/>

Example 1:

- Input: `nums = [1,1,1,2,2,3]`, `k = 2`
- Output: `[1,2]`

Example 2:

- Input: `nums = [1]`, `k = 1`
 - Output: `[1]`
-

Solution:

```
import java.util.*;
```

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        // Step 1: Build frequency map
        Map<Integer, Integer> freqMap = new HashMap<>();
        for (int num : nums) {
            freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);
        }

        // Step 2: Use a min-heap to keep top k frequent elements
        PriorityQueue<Map.Entry<Integer, Integer>> minHeap = new PriorityQueue<>(
            (a, b) -> a.getValue() - b.getValue()
        );

        // Step 3: Push elements into the heap
        for (Map.Entry<Integer, Integer> entry : freqMap.entrySet()) {
            minHeap.add(entry);
            if (minHeap.size() > k) {
                minHeap.poll(); // Remove the least frequent element
            }
        }
    }
}
```

```

// Step 4: Extract the top k elements from the heap
int[] result = new int[k];
int index = 0;
while (!minHeap.isEmpty()) {
    result[index++] = minHeap.poll().getKey();
}

return result;
}
}

```

Explanation:

- **Frequency Map Creation:**

- We first build a frequency map (`freqMap`), where the key is the element from the array, and the value is its frequency.

- **Min-Heap (Priority Queue):**

- We use a priority queue (min-heap) that stores map entries, where the smallest frequency elements come first.
- For every element in the frequency map, we add it to the heap. If the size of the heap exceeds `k`, we remove the least frequent element using `minHeap.poll()`. This ensures that the heap only contains the `k` most frequent elements at any time.

- **Extract Top K Elements:**

- Once the heap contains the `k` most frequent elements, we extract them and store them in the `result` array.

- **Time Complexity:** $O(n \log k)$ where `n` is the number of elements in `nums`, and `k` is the required top frequency count.

- **Space Complexity:** $O(n)$ for the frequency map and heap.

Problem 2: Kth Largest Element in an Array

Problem Statement:

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array. Note that it is the `k`th largest element in sorted order, not the `k`th distinct element.

Link to problem:

<https://leetcode.com/problems/kth-largest-element-in-an-array/description/>

Example 1:

- Input: nums = [3,2,1,5,6,4], k = 2
- Output: 5

Example 2:

- Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
- Output: 4

Solution:

```
import java.util.PriorityQueue;
```

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        // Step 1: Create a min-heap
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        // Step 2: Add elements to the heap
        for (int num : nums) {
            minHeap.add(num); // Add current element
            if (minHeap.size() > k) {
                minHeap.poll(); // Remove smallest if size exceeds k
            }
        }

        // Step 3: The top element is the kth largest
        return minHeap.peek();
    }
}
```

Explanation:

- **Create a Min-Heap:** We initialize a PriorityQueue (which acts as a min-heap by default in Java).
- **Add Elements to the Heap:** For each element in the array nums, we add it to the heap. If the heap size exceeds k, we remove the smallest element (minHeap.poll()). This ensures that we are left with the k largest elements in the heap.
- **Retrieve the kth Largest:**
After processing all the elements, the root of the heap (minHeap.peek()) contains the kth largest element, because the smallest element in the heap is the kth largest in the array.

-
- **Time Complexity:** $O(n \log k)$ where n is the number of elements in nums, and k is the required largest count.
 - **Space Complexity:** $O(k)$ for the heap.
-

